

LeetCode 546: Remove Boxes

Target Companies: Tencent, Capital One, Cisco

© Authored by Arpit Raj, LNMIIT Jaipur | Date: September 5, 2026

1. Problem Overview

Given an array containing numbers (representing colored boxes), at a time one can remove k continuous boxes of the same color and get $k \times k$ points. The objective is to find the maximum points you can get if boxes are removed optimally.

Example Walkthrough

Initial Array: `[1, 3, 2, 2, 2, 3, 4, 3, 1]`

- Remove all **2**s (3 boxes) → $3 \times 3 = 9$ points. Array becomes: `[1, 3, 3, 4, 3, 1]`
- Remove the **4** (1 box) → $1 \times 1 = 1$ point. Array becomes: `[1, 3, 3, 3, 1]`
- Remove all **3**s (3 boxes) → $3 \times 3 = 9$ points. Array becomes: `[1, 1]`
- Remove all **1**s (2 boxes) → $2 \times 2 = 4$ points. Array becomes: `[]`

Total Points: $9 + 1 + 9 + 4 = 23$

2. BTR Solution (Bottom-Up DP Approach)

The solution is conceptually similar to the "Burst Balloons" problem, but a standard 2D DP state $dp[l][r]$ is insufficient. Instead, we use a 3D DP state: $dp[l][r][k]$.

Understanding the State

Let k be the count of extra boxes having the same color and already attached to $boxes[l]$.

Range of k : $0, 1, 2, \dots, n-1$. The total number of values is n . Because at one point in the code we evaluate $k+1$, the maximum size required for the third dimension is $n+1$.

3. Transitions & Options

For a given segment $[l \dots r]$ with k extra boxes of the same color as $boxes[l]$, we have two main choices:

Option 1: Mix with Extra Boxes

Remove the k extra boxes along with $boxes[l]$ immediately. This merges $k+1$ boxes.

Points gained: $(k+1) \times (k+1)$

Next step: Process the remaining isolated segment $[l+1 \dots r]$ separately. Hence, we add $dp[l+1][r][0]$.

Option 2: Merge Later

Save $boxes[l]$ and merge it with another same-colored box at index m .

Visualizing the array: $[k \text{ extra boxes}] [l] [l+1 \dots m-1] [m \dots r]$

Since $boxes[l]$ and $boxes[m]$ have the same color, we want to merge them.

- Everything between l and m needs to be processed and removed: $dp[l+1][m-1][0]$.
- Now l and m are logically joined, meaning m inherits the k extra boxes plus $boxes[l]$.
Therefore, m gets one extra same-colored box, transitioning the state to: $dp[m][r][k+1]$.

4. C++ Implementation

The code strictly follows proper indentation, spacing, and utilizes the logic outlined above.

```

class Solution {
public:
    int removeBoxes(vector<int>& boxes) {
        int n=boxes.size();
        class="comment">//[k extra boxes with same colour as l]
[1.....r]

        vector<vector<vector<int>>>
dp(n,vector<vector<int>>(n,vector<int>(n+1,0)));
        class="comment">//[k can be 0.....n-1 , at max n-1 extra boxes ,
total n boxes he
        class="comment">//[k values can be 0,1,2,3,...n-1
        class="comment">//[so total n values
        class="comment">//[at one point in my code im doing k+1 , so max
value = n+1 , isliye n+1 kiya he

        for(int len=1;len<=n;len++){
            for(int l=0;l+len-1<n;l++){
                int r=l+len-1;

                for(int k=0;k<=n-1;k++){

                    class="comment">//[option 1 , mix with extra boxes
                    class="comment">//[k extra boxes] mix with [l] and then
[1+1.....r] alag se

                    class="comment">//[hence k+1 boxes merge

                    int ans=(k+1)*(k+1);

                    if(l+1<=r)
                        ans+=dp[l+1][r][0];

                    class="comment">//[option 2
                    class="comment">//[k extra box][l][1+1.....m-1]
[m.....r]

                    class="comment">//[l and m have same colour so we want to
merge them

                    class="comment">//[1+1.....m-1] needs to be processed
and removed

                    class="comment">//[k extra box][l][m.....r]

```

```

class="comment">//[k+1 extra box][m.....r]

for(int m=l+1;m<=r;m++){
    if(boxes[l]==boxes[m]){

        int curr=0;

        if(l+1<=m-1)
            curr+=dp[l+1][m-1][0];

        if(m<=r)
            curr+=dp[m][r][k+1];

        ans=max(ans,curr);
    }
}

dp[l][r][k]=ans;

    }
}

return dp[0][n-1][0];
}
};

```